COP290 Assignment 2: Task Board

Design Decisions Report

Arin Nitin Chandurkar 2024CS50002
Keshav Navandar 2024CS10441

March 22, 2026

GitHub Repository

The source code and version history for this project can be accessed at:
`https://github.com/ken7d61/website`

System Architecture & Bootstrap

- **Automatic Seeding:** On server startup, the system verifies and auto-provisions a "Global Admin" account (`admin@workspace.com`) if missing to ensure immediate system accessibility.
- **Restricted Capabilities:** To prevent catastrophic data loss, only the Global Admin possesses the system authority to create and delete entire projects.

Technology Stack & Data Management

- **Backend:** Developed using Node.js and TypeScript to ensure robust type safety across all controllers and modularity.
- **Database:** MySQL serves as the relational backbone. We utilize strict foreign key constraints with `ON DELETE CASCADE` to guarantee no orphaned tasks or comments are left behind.
- **Avatar Storage:** Optimized for low user volume, user avatars are stored directly as Base64 strings within `LONGTEXT` columns. Backend payload limits were explicitly increased to 1MB to accommodate this, removing the overhead of external file servers.

Workflow & Kanban Logic

- **Dynamic Columns:** Status stages are managed dynamically via a dedicated columns table rather than hardcoded enums, allowing for board-specific customization.
- **WIP Limits:** Capacity enforcement is strictly validated during 'Move' or 'Create' operations to prevent bottlenecks. Default limits are set to 3 for 'In Progress' and 1 for 'In Review'.
- **Transition Rules:** Adjacency movement is permitted by default. Backward or non-adjacent forward movements require explicit configuration in an `allowed_workflows` table for strict governance.

Task Hierarchy (Story & Sub-tasks)

- **Hierarchy:** A 'Story' functions as a parent task, encapsulating 'Task' and 'Bug' types as children.
- **Status Derivation:** A Story's status is dynamically derived using "Minimum Task Precedence," ensuring the parent reflects the state of the furthest-behind child task.
- **Structural Bounds:** Stories must be generated via a dedicated interface; standard tasks cannot be retroactively converted to Stories to maintain strict structural boundaries.

Security & Authentication

- **Dual JWT Strategy:** Employs short-lived Access Tokens (15 min) and long-lived Refresh Tokens (7 days) stored in `httpOnly` cookies to effectively neutralize XSS risks.
- **Cryptography:** Passwords are mathematically secured using `bcrypt` with a salt round of 10.
- **Silent Auth:** A frontend interceptor catches 401 Unauthorized responses and seamlessly triggers an `/auth/refresh` request to obtain a new token, ensuring a frictionless experience.

User Interface & UX Design

- **Styling:** Uses vanilla CSS with CSS Modules (`*.module.css`) to prevent class-name collisions without the overhead of heavily abstracted utility-class frameworks.
- **Role-Based Access (RBAC):** The UI proactively disables restricted actions (like assignee selection for Project Members) using tooltip feedback to prevent user frustration.
- **Synchronization:** Implements 30-second short polling for notification updates to maintain reliability while eliminating the connection-drop overhead of persistent sockets.

Audit & Reliability

- **Immutable Logs:** The audit log captures point-in-time snapshots of key states (e.g., `is_closed`) at the exact moment an event occurs to ensure historical accuracy regardless of future changes.
- **API Execution:** Frontend fetch calls are handled sequentially to act as a throttle, preventing database connection bottlenecking and ensuring a predictable loading waterfall.